

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
```

```
(X_train, y_train), (X_test, y_test) = datasets.mnist.load_data()

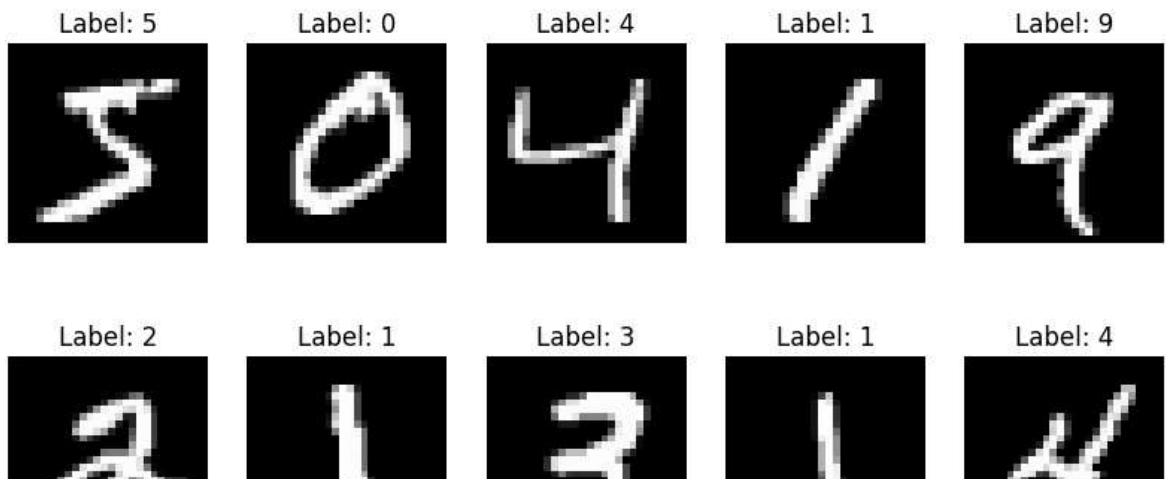
print("Training Images:", X_train.shape)
print("Testing Images:", X_test.shape)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-data/11490434/11490434> — 0s 0us/step
 Training Images: (60000, 28, 28)
 Testing Images: (10000, 28, 28)

```
plt.figure(figsize=(10,5))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title(f"Label: {y_train[i]}")
    plt.axis('off')

plt.show()
```



```
X_train = X_train / 255.0
X_test = X_test / 255.0

X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)

print(X_train.shape)

(60000, 28, 28, 1)
```

```

model = models.Sequential([

    layers.Conv2D(32,(3,3),activation='relu',
                  input_shape=(28,28,1)),

    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64,(3,3),activation='relu'),

    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64,(3,3),activation='relu'),

    layers.Flatten(),

    layers.Dense(64,activation='relu'),

    layers.Dense(10,activation='softmax')
])

model.summary()

```

```

/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv2d.py:100:
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_2 (Conv2D)	(None, 3, 3, 64)	36,928
flatten (Flatten)	(None, 576)	0
dense (Dense)	(None, 64)	36,928
dense_1 (Dense)	(None, 10)	650

Total params: 93,322 (364.54 KB)
Trainable params: 93,322 (364.54 KB)
Non-trainable params: 0 (0.00 B)

```

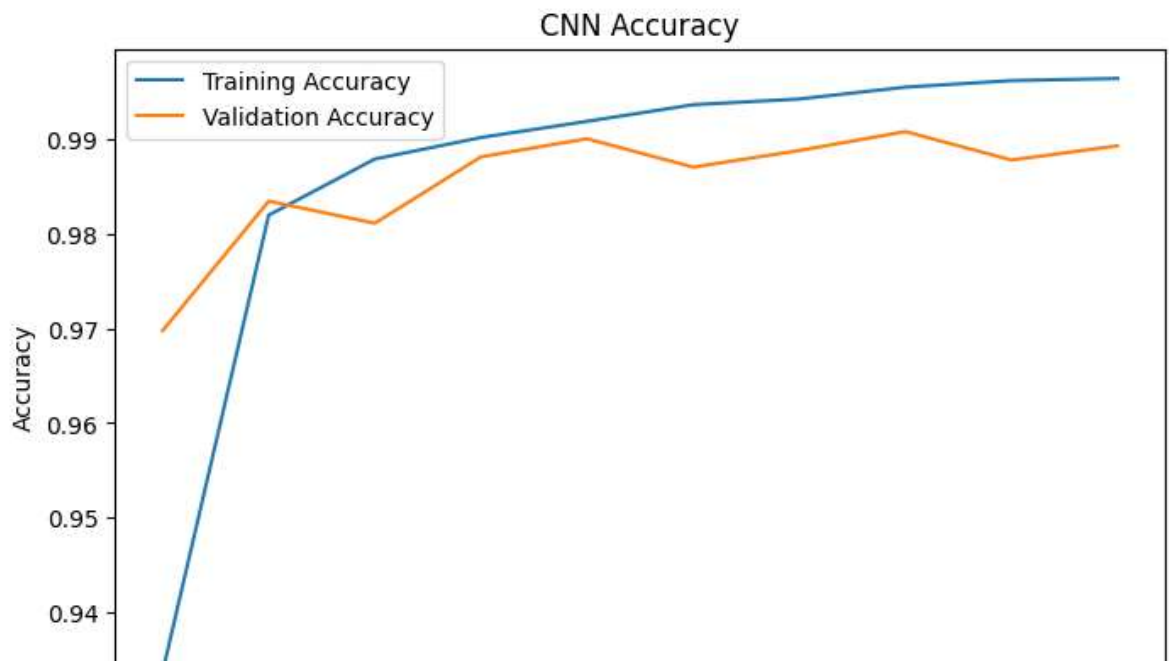
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

```

```
history = model.fit(  
    X_train,  
    y_train,  
    epochs=10,  
    validation_split=0.2,  
    batch_size=64  
)
```

```
3p - accuracy: 0.9337 - loss: 0.2176 - val_accuracy: 0.9697 - val_loss: 0.1024  
3p - accuracy: 0.9819 - loss: 0.0577 - val_accuracy: 0.9834 - val_loss: 0.0576  
3p - accuracy: 0.9879 - loss: 0.0383 - val_accuracy: 0.9811 - val_loss: 0.0646  
3p - accuracy: 0.9901 - loss: 0.0317 - val_accuracy: 0.9881 - val_loss: 0.0389  
3p - accuracy: 0.9918 - loss: 0.0255 - val_accuracy: 0.9900 - val_loss: 0.0379  
3p - accuracy: 0.9936 - loss: 0.0200 - val_accuracy: 0.9870 - val_loss: 0.0461  
3p - accuracy: 0.9942 - loss: 0.0167 - val_accuracy: 0.9887 - val_loss: 0.0419  
3p - accuracy: 0.9954 - loss: 0.0135 - val_accuracy: 0.9908 - val_loss: 0.0373  
3p - accuracy: 0.9961 - loss: 0.0120 - val_accuracy: 0.9877 - val_loss: 0.0513  
3p - accuracy: 0.9964 - loss: 0.0108 - val_accuracy: 0.9893 - val_loss: 0.0474
```

```
plt.figure(figsize=(8,5))  
  
plt.plot(history.history['accuracy'],  
         label='Training Accuracy')  
  
plt.plot(history.history['val_accuracy'],  
         label='Validation Accuracy')  
  
plt.xlabel('Epoch')  
plt.ylabel('Accuracy')  
plt.legend()  
plt.title("CNN Accuracy")  
plt.show()
```



```
loss, accuracy = model.evaluate(X_test, y_test)
```

```
print("Test Accuracy:", accuracy)
```

313/313 ————— 3s 9ms/step - accuracy: 0.9901 - loss: 0.0346
Test Accuracy: 0.9901000261306763

```
predictions = model.predict(X_test)
```

```
predicted_labels = np.argmax(predictions, axis=1)
```

```
print("First 10 Predictions:")  
print(predicted_labels[:10])
```

```
print("Actual Labels:")  
print(y_test[:10])
```

313/313 ————— 3s 11ms/step
First 10 Predictions:
[7 2 1 0 4 1 4 9 5 9]
Actual Labels:
[7 2 1 0 4 1 4 9 5 9]

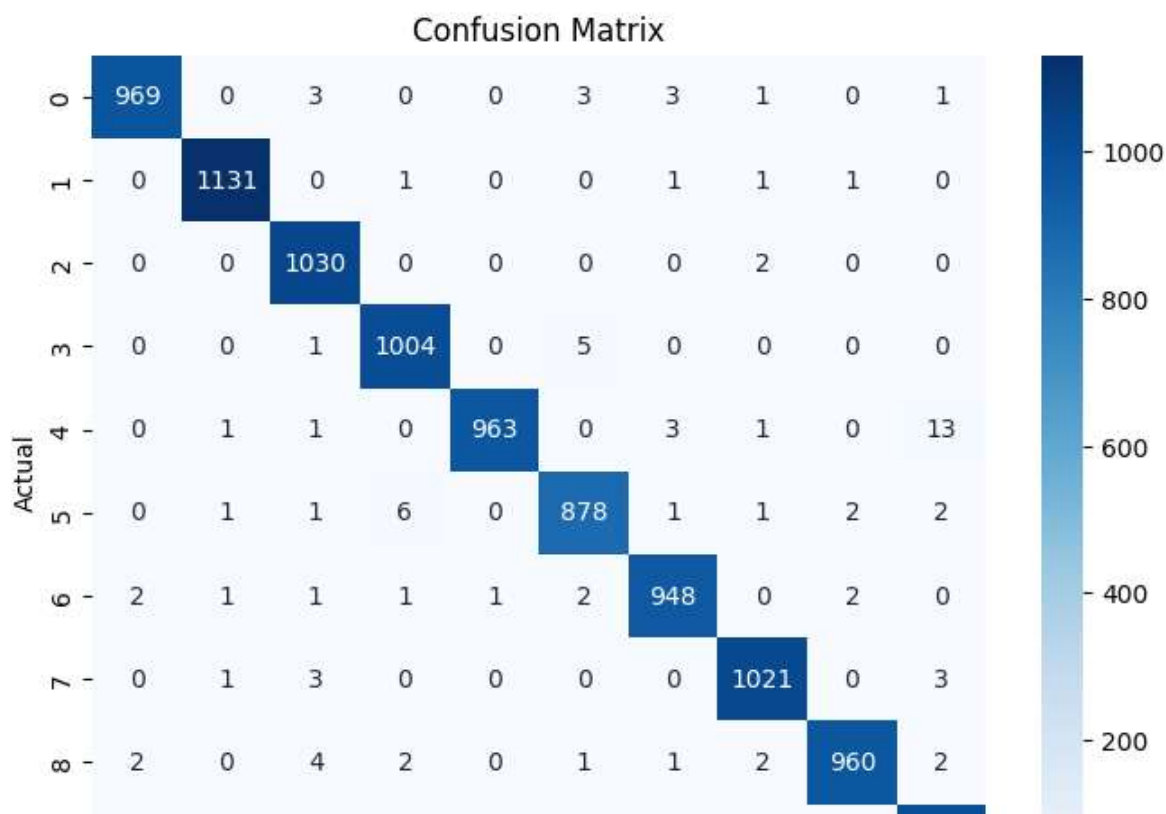
```
cm = confusion_matrix(y_test, predicted_labels)
```

```
plt.figure(figsize=(8,6))
```

```
sns.heatmap(cm,  
            annot=True,  
            fmt='d',  
            cmap='Blues')
```

```
plt.xlabel("Predicted")  
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix")
plt.show()
```



```
print(classification_report(
    y_test,
    predicted_labels
))
```

	precision	recall	f1-score	support
0	1.00	0.99	0.99	980
1	1.00	1.00	1.00	1135
2	0.99	1.00	0.99	1032
3	0.99	0.99	0.99	1010
4	1.00	0.98	0.99	982
5	0.98	0.98	0.98	892
6	0.99	0.99	0.99	958
7	0.99	0.99	0.99	1028
8	0.99	0.99	0.99	974
9	0.98	0.99	0.98	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

```
plt.figure(figsize=(12,6))

for i in range(10):

    plt.subplot(2,5,i+1)

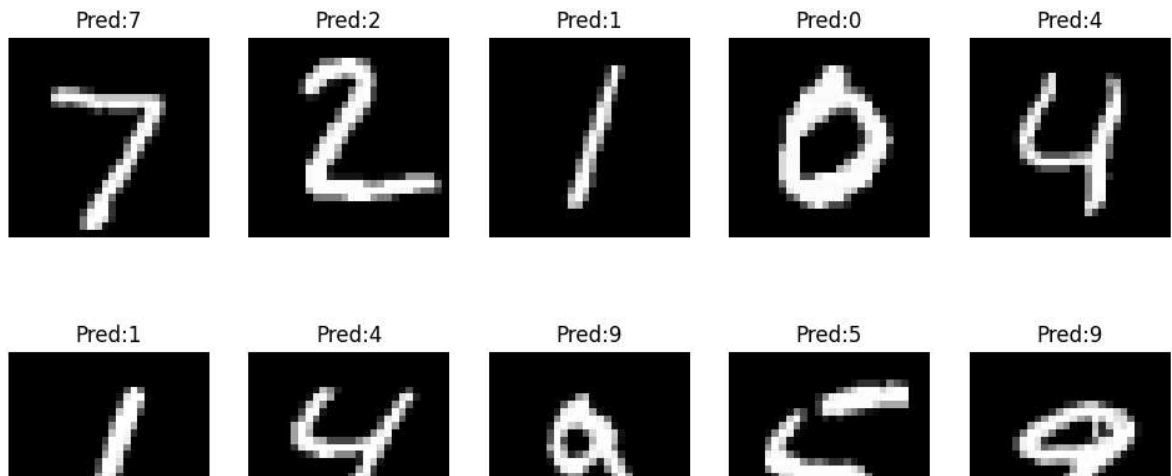
    plt.imshow(X_test[i].reshape(28,28),
```

```
cmap= gray )

plt.title(
    f"Pred:{predicted_labels[i]}"
)

plt.axis('off')

plt.show()
```



```
print("""
Observation:

1. CNN successfully classified handwritten digits.
2. Model achieved approximately 98-99% accuracy.
3. Convolution layers extracted image features.
4. Pooling layers reduced dimensionality.
5. Softmax layer predicted digit classes.
6. Confusion matrix showed very few misclassifications.
""")
```